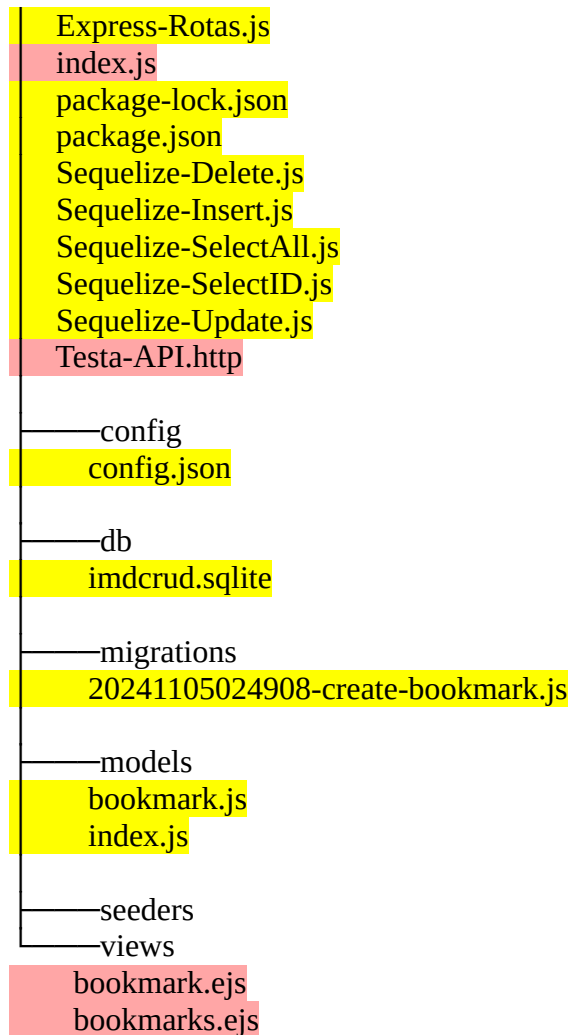


FASE 04 – API REST COM EJS

Nas fases anteriores, geramos os arquivos tarjados em amarelo. Revise e especifique qual foi o papel de cada um desses arquivos:

C:.



Nessa fase, finalizaremos a API REST com o CRUD dos dados nas rotas /api (post, get, put, delete) e rotas /bookmarks e /bookmark/id com os resultados em layout (formatados html/css):

01. Crie o EJS **bookmark.ejs** para exibir os dados de um bookmark, conforme modelo:

The screenshot shows a web browser window with the address bar displaying 'localhost:8080/bookmark/2'. The page title is 'Bookmark'. The form contains the following data:

Field	Value
Id	2
Site	GitHub
Detalhes	Repositório para controle de versão e colaboração
Link	https://github.com

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Bookmark</title>
  <style>
    h1 {
      text-align: center;
    }
    /* Estilos para o formulário */
    form {
      max-width: 400px;
      margin: auto;
    }
    /* Container para cada linha do formulário */
    .form-group {
      display: flex;
      margin-bottom: 10px;
    }
    /* Alinha os rótulos à direita */
    label {
      width: 100px;
      text-align: right;
      margin-right: 10px;
    }
    /* Define largura para os campos de entrada */
    input[type="text"],
    textarea {
      flex: 1;
      padding: 5px;
    }
    /* Formatação do textarea */
    textarea {
      resize: vertical;
      height: 80px;
    }
  </style>
</head>
<body>
  <h1>Bookmark</h1>
  <div>
    <div class="form-group">
      <label>Id:</label>
      <input type="text" value="2"/>
    </div>
    <div class="form-group">
      <label>Site:</label>
      <input type="text" value="GitHub"/>
    </div>
    <div class="form-group">
      <label>Detalhes:</label>
      <textarea value="Repositório para controle de versão e colaboração"/>
    </div>
    <div class="form-group">
      <label>Link:</label>
      <input type="text" value="https://github.com"/>
    </div>
  </div>
</body>
</html>
```

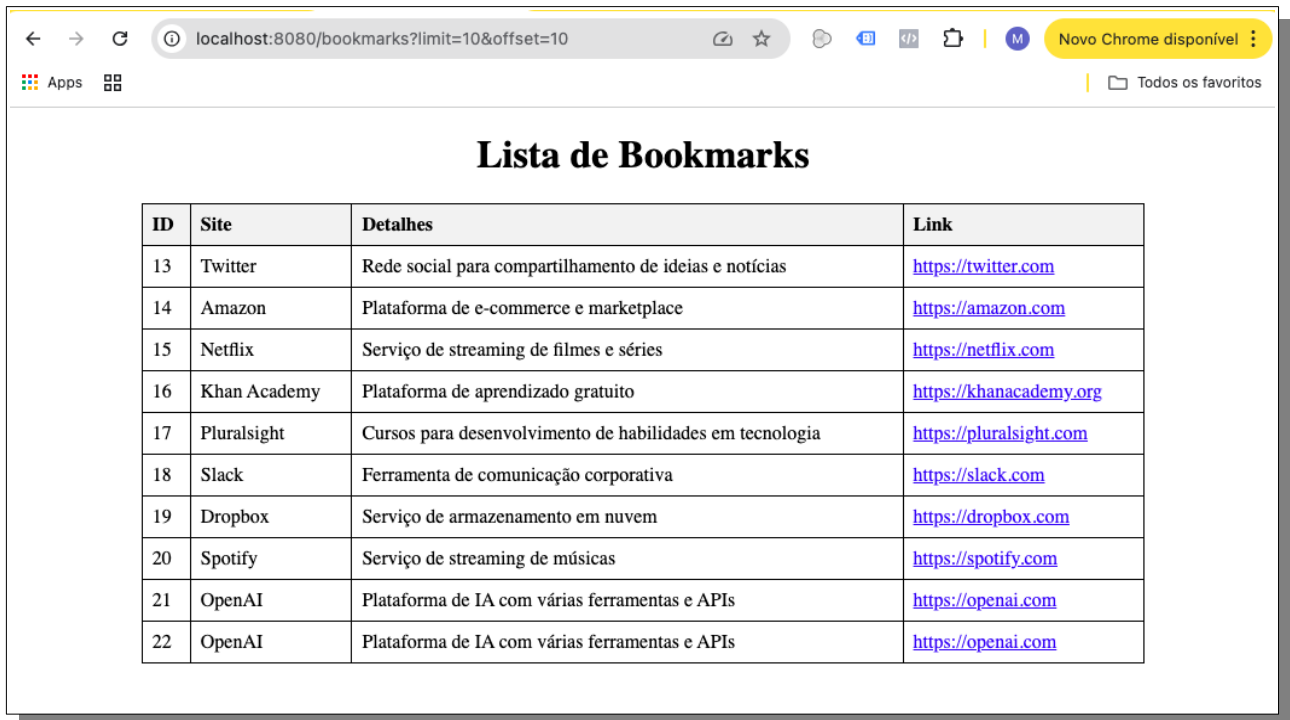
```
    }
  </style>
</head>
<body>
  <h1>Bookmark</h1>
  <form>
    <div class="form-group">
      <label for="Id">Id:</label>
      <input type="number" id="id" name="id" value="<%= bookmark.id %>">
    </div>

    <div class="form-group">
      <label for="site">Site:</label>
      <input type="text" id="site" name="site" value="<%= bookmark.site %>">
    </div>

    <div class="form-group">
      <label for="detalhes">Detalhes:</label>
      <textarea id="detalhes" name="detalhes"><%= bookmark.detalhes %></textarea>
    </div>

    <div class="form-group">
      <label for="link">Link:</label>
      <input type="text" id="link" name="link" value="<%= bookmark.link %>">
    </div>
  </form>
</body>
</html>
```

02. Crie o EJS **bookmarks.ejs** para exibir os dados de um grupo de bookmarks (passando a quantidade – limit e o deslocamento - offset), conforme modelo:



ID	Site	Detalhes	Link
13	Twitter	Rede social para compartilhamento de ideias e notícias	https://twitter.com
14	Amazon	Plataforma de e-commerce e marketplace	https://amazon.com
15	Netflix	Serviço de streaming de filmes e séries	https://netflix.com
16	Khan Academy	Plataforma de aprendizado gratuito	https://khanacademy.org
17	Pluralsight	Cursos para desenvolvimento de habilidades em tecnologia	https://pluralsight.com
18	Slack	Ferramenta de comunicação corporativa	https://slack.com
19	Dropbox	Serviço de armazenamento em nuvem	https://dropbox.com
20	Spotify	Serviço de streaming de músicas	https://spotify.com
21	OpenAI	Plataforma de IA com várias ferramentas e APIs	https://openai.com
22	OpenAI	Plataforma de IA com várias ferramentas e APIs	https://openai.com

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Bookmarks</title>
  <style>
    h1 {
      text-align: center;
    }
    table {
      width: 80%;
      margin-left: auto;
      margin-right: auto;
      border-collapse: collapse;
    }
    table, th, td {
      border: 1px solid black;
    }
    th, td {
      padding: 8px;
      text-align: left;
    }
    th {
      background-color: #f2f2f2;
    }
  </style>
</head>
<body>

  <h1>Lista de Bookmarks</h1>
```

```
<table>
  <thead>
    <tr>
      <th>ID</th>
      <th>Site</th>
      <th>Detalhes</th>
      <th>Link</th>
    </tr>
  </thead>
  <tbody>
    <% bookmarks.forEach(function(bookmark) { %>
      <tr>
        <td><%= bookmark.id %></td>
        <td><%= bookmark.site %></td>
        <td><%= bookmark.detalhes %></td>
        <td><a href="<%= bookmark.link %>" target="_blank">
          <%= bookmark.link %></a></td>
      </tr>
    <% }) %>
  </tbody>
</table>

</body>
</html>
```

03. Crie o programa `index.ejs` para tratar as rotas:

```
// index.js
const express = require('express');
const { Sequelize, DataTypes } = require('sequelize');
const { Bookmark } = require('./models'); // Importa o modelo Bookmark

const app = express();
const PORT = 8080;

// Configura o diretório de views para o EJS
app.set('view engine', 'ejs');
app.set('views', './views');
app.use(express.json());

app.post('/api/bookmark', async (req, res) => {
  try {

    // Insere bookmark
    const createdData = req.body;

    const bookmark = await Bookmark.create(createdData);

    if (bookmark) {
      res.status(200).send(`Bookmark ${bookmark.id} inserido`);
    } else {
      res.status(404).send('Falha na criação do Bookmark');
    }
  } catch (error) {
    console.error('Erro ao criar bookmark:', error);
    res.status(500).send('Erro ao criar bookmark');
  }
});

// Configura a rota /bookmarks para exibir os dados da tabela Bookmark
app.get('/bookmarks', async (req, res) => {
  try {
    // Obtém os valores de limite e deslocamento dos parâmetros da query
    const limit = parseInt(req.query.limit) || 100; // Padrão de 100 registros se não especificado
    const offset = parseInt(req.query.offset) || 0; // Padrão de 0 se não especificado

    // Lê registros da tabela Bookmark usando Sequelize com limit e offset
    const bookmarks = await Bookmark.findAll({
      limit: limit,
      offset: offset
    });

    // Renderiza a página EJS passando os dados para o template
    res.render('bookmarks', { bookmarks });
  } catch (error) {
    console.error('Erro ao buscar bookmarks:', error);
    res.status(500).send('Erro ao buscar bookmarks');
  }
});
```

```
});
```

```
app.get('/api/bookmarks', async (req, res) => {
  try {
    // Obtém os valores de limite e deslocamento dos parâmetros da query
    const limit = parseInt(req.query.limit) || 100; // Padrão de 100 registros se não especificado
    const offset = parseInt(req.query.offset) || 0; // Padrão de 0 se não especificado

    // Lê registros da tabela Bookmark usando Sequelize com limit e offset
    const bookmarks = await Bookmark.findAll({
      limit: limit,
      offset: offset
    });
    res.status(200).send(bookmarks);
  } catch (error) {
    console.error('Erro ao buscar bookmarks:', error);
    res.status(500).send('Erro ao buscar bookmarks');
  }
});
```

```
app.get('/bookmark/:id', async (req, res) => {
  try {
    // Busca o bookmark pelo ID fornecido na URL
    const bookmark = await Bookmark.findByPk(req.params.id);

    if (bookmark) {
      // Renderiza o formulário EJS passando os dados do bookmark
      res.render('bookmark', { bookmark });
    } else {
      res.status(404).send('Bookmark não encontrado');
    }
  } catch (error) {
    console.error('Erro ao buscar bookmark:', error);
    res.status(500).send('Erro ao buscar bookmark');
  }
});
```

```
app.get('/api/bookmark/:id', async (req, res) => {
  try {
    // Busca o bookmark pelo ID fornecido na URL
    const bookmark = await Bookmark.findByPk(req.params.id);

    if (bookmark) {
      res.status(200).send(bookmark);
    } else {
      res.status(404).send('Bookmark não encontrado');
    }
  } catch (error) {
    console.error('Erro ao buscar bookmark:', error);
    res.status(500).send('Erro ao buscar bookmark');
  }
});
```

```
app.put('/api/bookmark/:id', async (req, res) => {
  try {
    // Atualiza o bookmark pelo ID fornecido na URL
    const updatedData = req.body;

    const [affectedRows] = await Bookmark.update(
      updatedData,          // Dados a serem atualizados
      { where: { id: req.params.id } } // Condição para selecionar o registro
    );

    if (affectedRows>0) {
      res.status(200).send(`Bookmark ${req.params.id} atualizado`);
    } else {
      res.status(404).send('Bookmark não encontrado');
    }
  } catch (error) {
    console.error('Erro ao buscar bookmark:', error);
    res.status(500).send('Erro ao buscar bookmark');
  }
});
```

```
app.delete('/api/bookmark/:id', async (req, res) => {
  try {
    // Apaga o bookmark pelo ID fornecido na URL

    const affectedRows = await Bookmark.destroy({
      where: { id: req.params.id } // Condição para selecionar o registro
    });

    if (affectedRows>0) {
      res.status(200).send(`Bookmark ${req.params.id} apagado`);
    } else {
      res.status(404).send('Bookmark não encontrado');
    }
  } catch (error) {
    console.error('Erro ao buscar bookmark:', error);
    res.status(500).send('Erro ao buscar bookmark');
  }
});
```

```
// Inicia o servidor
app.listen(PORT, () => {
  console.log(`Servidor rodando em http://localhost:${PORT}`);
});
```


04. Teste todas as rotas com as chamadas http – crie o arquivo **testaAPI.http**:

POST http://localhost:8080/api/bookmark

Content-Type: application/json

```
{
  "id": 102,
  "site": "Site do IMD",
  "detalhes": "Portal IMD para acesso a plataforma de cursos",
  "link": "https://imdtec.imd.ufrn.br/"
}
```

###

GET http://localhost:8080/api/bookmark/102

###

GET http://localhost:8080/api/bookmarks

###

GET http://localhost:8080/bookmark/2

###

GET http://localhost:8080/bookmarks

###

PUT http://localhost:8080/api/bookmark/2

Content-Type: application/json

```
{
  "site": "Site do IMD",
  "detalhes": "Portal IMD para acesso a plataforma de cursos",
  "link": "https://imdtec.imd.ufrn.br/"
}
```

###

PUT http://localhost:8080/api/bookmark/2

Content-Type: application/json

```
{
  "site": "Stack Overflow"
}
```

###

DELETE http://localhost:8080/api/bookmark/102

###

GET http://localhost:8080/api/bookmark/2

###

NOTA: É importante você testar TODAS as rotas da API alterando os parâmetros e entendendo toda aplicação